SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Code Challenge (High)
Assessment Tool Weightage: 35%

Dr

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	**5**
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	**5**
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	**5**
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	**5**
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	**5**
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	**5**
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	**5**
8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	**5**
9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	**5**

10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	**5**

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

1. **Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.**

Program :

```python
MAX = 100

heap = []


def insert_record():
    if len(heap) == MAX:
        print("Heap file is full!")
        return

    record_id = int(input("Enter ID: "))
    name = input("Enter Name: ")

    heap.append({
        "id": record_id,
        "name": name,
        "deleted": False
    })

    print("Record inserted successfully.")


def delete_record():
    record_id = int(input("Enter ID to delete: "))

    for record in heap:
        if record["id"] == record_id and not record["deleted"]:
            record["deleted"] = True
```

Sample Output :

```
--- Heap File Organization ---
1. Insert Record
2. Delete Record
3. Sequential Scan
4. Exit
Enter your choice: 1
Enter ID: 102
Enter Name: Kavin
Record inserted successfully.

--- Heap File Organization ---
1. Insert Record
2. Delete Record
3. Sequential Scan
4. Exit
Enter your choice: 3

Heap File Records:
ID: 101          Name: Arun
ID: 102          Name: Kavin
```

2. **Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.**

Program :

```
TABLE_SIZE = 50

hash_table = [[] for _ in range(TABLE_SIZE)]


def hash_function(student_id):
    return student_id % TABLE_SIZE


def insert_student():
    student_id = int(input("Enter Student ID: "))
    name = input("Enter Student Name: ")

    index = hash_function(student_id)

    hash_table[index].append((student_id, name))

    print("Student record inserted successfully.")


def search_student():
    student_id = int(input("Enter Student ID to search: "))

    index = hash_function(student_id)

    for record in hash_table[index]:
        if record[0] == student_id:
            print("Student ID:", record[0])
            print("Student Name:", record[1])
```

Sample Output :

```
--- Static Hashing ---
1. Insert Student
2. Search Student
3. Display Hash Table
4. Exit
Enter your choice: 1
Enter Student ID: 101
Enter Student Name: Arun
Student record inserted successfully.

--- Static Hashing ---
1. Insert Student
2. Search Student
3. Display Hash Table
4. Exit
Enter your choice: 1
Enter Student ID: 204
Enter Student Name: Syed
Student record inserted successfully.

--- Static Hashing ---
1. Insert Student
2. Search Student
3. Display Hash Table
4. Exit
Enter your choice: 3

Hash Table:
1 -> (101, 'Arun') -> None
4 -> (204, 'Syed') -> None
```

3. **Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.**

Program :

```
records = []


def insert_record():
    student_id = int(input("Enter Student ID: "))
    name = input("Enter Student Name: ")

    records.append((student_id, name))
    records.sort()

    print("Record inserted successfully.")


def binary_search():
    student_id = int(input("Enter Student ID to search: "))

    low = 0
    high = len(records) - 1

    while low <= high:
        mid = (low + high) // 2

        if records[mid][0] == student_id:
            print("Record found!")
            print("Student ID:", records[mid][0])
            print("Student Name:", records[mid][1])
            return

        elif records[mid][0] < student_id:
            low = mid + 1
```

Sample Output:

```
--- Sorted File Organization ---
1. Insert Record
2. Binary Search
3. Display Records
4. Exit
Enter your choice: 1
Enter Student ID: 124
Enter Student Name: Arun
Record inserted successfully.

--- Sorted File Organization ---
1. Insert Record
2. Binary Search
3. Display Records
4. Exit
Enter your choice: 1
Enter Student ID: 145
Enter Student Name: Gautham
Record inserted successfully.

--- Sorted File Organization ---
1. Insert Record
2. Binary Search
3. Display Records
4. Exit
Enter your choice: 2
Enter Student ID to search: 124
Record found!
Student ID: 124
Student Name: Arun
```

4. **Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.**

Program:

```
class BTreeNode:
    def __init__(self, leaf=False):
        self.keys = []
        self.children = []
        self.leaf = leaf


class BTree:
    def __init__(self):
        self.root = BTreeNode(True)

    def search(self, node, key):
        i = 0

        while i < len(node.keys) and key > node.keys[i]:
            i += 1

        if i < len(node.keys) and key == node.keys[i]:
            return True

        if node.leaf:
            return False

        return self.search(node.children[i], key)

```

Sample Output:

```
B-Tree:
[10, 20]
  [5, 6, 7]
  [12, 17]
  [30]

Enter key to search: 17
Key found.
```

5. **Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.**

Program :

```
class BPlusTree:
    def __init__(self, order=4):
        self.order = order
        self.leaves = []
        self.data = {}

    def insert(self, key):
        self.data[key] = key

        if key not in self.leaves:
            self.leaves.append(key)
            self.leaves.sort()

    def search(self, key):
        if key in self.data:
            return self.data[key]
        return None

    def range_query(self, start, end):
        result = []

        for key in self.leaves:
            if start <= key <= end:
                result.append(key)

```

Sample Output :

```
Leaf Level:
5 -> 6 -> 7 -> 10 -> 12 -> 17 -> 20 -> 25 -> 30 -> 35

Enter key to search: 17
Key found: 17

Enter range start: 10
Enter range end: 25
Keys in range: [10, 12, 17, 20, 25]
```

6. **Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.**

Program :

```
records = [
    (101, "Arun"),
    (102, "Kumar"),
    (103, "Ravi"),
    (104, "Kavin"),
    (105, "Ajay")
]

index = []


def build_index():
    index.clear()

    for position, record in enumerate(records):
        index.append((record[0], position))


def search():
    key = int(input("Enter Student ID to search: "))

    low = 0
    high = len(index) - 1

    while low <= high:
```

Sample Output :

```
--- Dense Primary Index ---
1. Search
2. Display Index
3. Display Records
4. Exit
Enter your choice: 2

Dense Primary Index:
Index Key          Record Position
101                 0
102                 1
103                 2
104                 3
105                 4

--- Dense Primary Index ---
1. Search
2. Display Index
3. Display Records
4. Exit
Enter your choice: 1
Enter Student ID to search: 104
Record found!
Student ID: 104
Student Name: Kavin
```

7. **Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.**

Program :

```
class Bucket:
    def __init__(self, depth):
        self.depth = depth
        self.records = []


class ExtendibleHashing:
    def __init__(self, bucket_size=2):
        self.global_depth = 1
        self.bucket_size = bucket_size

        bucket0 = Bucket(1)
        bucket1 = Bucket(1)

        self.directory = [bucket0, bucket1]

    def hash_function(self, key):
        return key & ((1 << self.global_depth) - 1)

    def double_directory(self):
        old_directory = self.directory
        self.directory = old_directory + old_directory
        self.global_depth += 1

    def split_bucket(self, index):
```

Sample Output :

```
Inserting: 1

Directory:
0 -> [] (Local Depth: 1 )
1 -> [1] (Local Depth: 1 )

Inserting: 3

Directory:
0 -> [] (Local Depth: 1 )
1 -> [1, 3] (Local Depth: 1 )

Inserting: 5

Overflow occurred while inserting: 5
Directory doubling...
Global Depth: 2

Directory:
00 -> [] (Local Depth: 1 )
01 -> [1, 5] (Local Depth: 2 )
10 -> [] (Local Depth: 1 )
11 -> [3] (Local Depth: 2 )

Inserting: 7

Directory:
00 -> [] (Local Depth: 1 )
01 -> [1, 5] (Local Depth: 2 )
10 -> [] (Local Depth: 1 )
11 -> [3, 7] (Local Depth: 2 )
```

8. Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.

Program :

```python
import math

total_records = 100
records_per_block = 10

total_blocks = math.ceil(total_records / records_per_block)

print("Total Records:", total_records)
print("Records per Block:", records_per_block)
print("Total Blocks:", total_blocks)

record = int(input("\nEnter record number to retrieve: "))

if record < 1 or record > total_records:
    print("Invalid record number.")
else:
    sequential_io = math.ceil(record / records_per_block)

    indexed_io = 1 + 1

    print("\n--- Retrieval Results ---")
    print("Record:", record)
    print("Block containing record:", math.ceil(record / records_per_block))
    print("Sequential Retrieval I/O:", sequential_io)
    print("Indexed Retrieval I/O:", indexed_io)

    if sequential_io > indexed_io:
        print("Indexed retrieval is faster.")
```

Sample Output:

```
Total Records: 100
Records per Block: 10
Total Blocks: 10

Enter record number to retrieve: 75

--- Retrieval Results ---
Record: 75
Block containing record: 8
Sequential Retrieval I/O: 8
Indexed Retrieval I/O: 2
Indexed retrieval is faster.
```

9. Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.

Program :

```
import time

records = [(i, f"Student_{i}") for i in range(1, 10001)]

dense_index = []
sparse_index = []

for position, record in enumerate(records):
    dense_index.append((record[0], position))

for position in range(0, len(records), 10):
    sparse_index.append((records[position][0], position))


def dense_search(key):
    low = 0
    high = len(dense_index) - 1

    while low <= high:
        mid = (low + high) // 2

        if dense_index[mid][0] == key:
            return records[dense_index[mid][1]]

        elif dense_index[mid][0] < key:
            low = mid + 1

        else:
            high = mid - 1

```

Sample Output :

```
Enter Student ID to search: 7850


--- Search Results ---
Dense Index Result: (7850, 'Student_7850')
Sparse Index Result: (7850, 'Student_7850')


--- Timing Analysis ---
Dense Index Search Time : 5.570007488131523e-05 seconds
Sparse Index Search Time: 3.7800054997205734e-05 seconds
```

10. Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.

Program :

```
class BPlusTree:
    def __init__(self, order=4):
        self.order = order
        self.keys = []

    def insert(self, key, record):
        self.keys.append((key, record))
        self.keys.sort()

    def traverse(self):
        print("\nB+ Tree Leaf Level:")

        for key, record in self.keys:
            print(f"{key} -> {record}")

    def range_query(self, start, end):
        print(f"\nRecords between {start} and {end}:")

        found = False

        for key, record in self.keys:
            if start <= key <= end:
                print(f"Key: {key}, Record: {record}")
                found = True

```

Sample Output:

```
B+ Tree Leaf Level:
10 -> Student_10
15 -> Student_15
20 -> Student_20
25 -> Student_25
30 -> Student_30
35 -> Student_35
40 -> Student_40
45 -> Student_45
50 -> Student_50

Records between 15 and 45:
Key: 15, Record: Student_15
Key: 20, Record: Student_20
Key: 25, Record: Student_25
Key: 30, Record: Student_30
Key: 35, Record: Student_35
Key: 40, Record: Student_40
Key: 45, Record: Student_45
```